

Data Science Scraping Project - Code Appendix

File: Harvester.py

```
from curl_cffi import requests
from bs4 import BeautifulSoup
import pandas as pd
import time
import random

# --- CONFIGURATION ---
# 1. URL for the 2021-2022 Season
url = "https://fbref.com/en/comps/11/2021-2022/schedule/2021-2022-Serie-A-Scores-and-Fixtures"

# 2. Output File for 2021-2022 Links
output_file = 'SerieA_links_2021.csv'
# -----

def scrape_with_retry(url, max_retries=3):
    """Tries to scrape a URL, pauses and retries if blocked."""
    for attempt in range(max_retries):
        try:
            print(f"Fetching (Attempt {attempt+1}): {url}")
            # UPDATED: Use 'safari15_3' to match your Worker script
            response = requests.get(url, impersonate="safari15_3", timeout=15)

            if response.status_code == 200:
                return response
            elif response.status_code in [403, 429]:
                # If blocked, wait 60 seconds and try again
                print(f"!!! Blocked (403/429). Waiting 60s before Retry...")
                time.sleep(60 + random.uniform(1, 10))
            else:
                print(f"Error: {response.status_code}")
                return None
        except Exception as e:
            print(f"Connection Error: {e}. Retrying...")
            time.sleep(10)

    print(f"Failed after {max_retries} attempts.")
    return None

# --- MAIN EXECUTION ---
response = scrape_with_retry(url)

if response and response.status_code == 200:
    soup = BeautifulSoup(response.content, 'html.parser')

    # Find the links
    match_links = []
```

Data Science Scraping Project - Code Appendix

```
report_cells = soup.find_all('td', {'data-stat': 'match_report'})

for cell in report_cells:
    link_tag = cell.find('a')
    if link_tag and link_tag.text == "Match Report":
        href = link_tag.get('href')
        full_link = f"https://fbref.com{href}"
        match_links.append(full_link)

# Save to CSV
if len(match_links) > 0:
    print(f"Success! Found {len(match_links)} matches.")
    df = pd.DataFrame(match_links, columns=['Match_URL'])
    df.to_csv(output_file, index=False)
    print(f"Links saved to '{output_file}'. Check your folder!")
else:
    print("Error: Found 0 links. The page might have loaded but the structure is different.")

else:
    print("Could not retrieve links even after retries.")
```

Data Science Scraping Project - Code Appendix

File: worker.py

```
from curl_cffi import requests
from bs4 import BeautifulSoup
import pandas as pd
import re
import time
import random

# --- CONFIGURATION ---
# TARGETING SERIE A 2021
input_file = 'SerieA_links_2021.csv'
output_file = 'Final_SerieA_DataSet_2021.csv'
# -----

# --- TACTICAL SCALE (6-Point) ---
POSITION_VALUE = {
    "GK": 1,
    "DF": 2, "FB": 2, "LB": 2, "RB": 2, "CB": 2,
    "WB": 3, "DM": 3,
    "CM": 4, "MF": 4,
    "AM": 5, "LM": 5, "RM": 5,
    "FW": 6, "LW": 6, "RW": 6, "ST": 6, "SS": 6,
    "Unknown": 0
}

def get_match_metadata(soup, url):
    season = "Unknown"
    title_tag = soup.find('h1')
    if title_tag:
        title_text = title_tag.get_text()
        year_match = re.search(r'20\d{2}', title_text)
        if year_match:
            season = year_match.group(0)

    matchweek = "0"
    mw_element = soup.find(string=re.compile(r'Matchweek \d+'))
    if mw_element:
        mw_num = re.search(r'Matchweek (\d+)', mw_element)
        if mw_num:
            matchweek = mw_num.group(1)
    else:
        matchweek = "Cup"

    try:
        game_uuid = url.split('/matches/')[1].split('/')[0]
    except:
        game_uuid = "unknown"
```

Data Science Scraping Project - Code Appendix

```
    return season, matchweek, game_uuid

def build_player_map(soup):
    player_positions = {}
    tables = soup.find_all('table', id=lambda x: x and 'summary' in x and 'stats_' in x)
    for table in tables:
        rows = table.find_all('tr')
        for row in rows:
            name_cell = row.find('th', {'data-stat': 'player'})
            pos_cell = row.find('td', {'data-stat': 'position'})
            if name_cell and pos_cell:
                name = name_cell.get_text(strip=True)
                pos_text = pos_cell.get_text(strip=True)
                first_pos = pos_text.split(',')[0] if pos_text else "Unknown"
                player_positions[name] = first_pos
    return player_positions

def determine_sub_type(pos_off, pos_on):
    val_off = POSITION_VALUE.get(pos_off, 2)
    val_on = POSITION_VALUE.get(pos_on, 2)

    if val_on > val_off:
        return 1, 0 # Offensive
    elif val_on < val_off:
        return 0, 1 # Defensive
    else:
        return 0, 0 # Neutral

def parse_events(soup, player_map, game_type, custom_game_id, season, matchweek):
    events_data = []
    events_wrap = soup.find('div', id='events_wrap')
    if not events_wrap:
        return []

    events = events_wrap.find_all('div', class_='event')

    current_home_score = 0
    current_away_score = 0
    home_reds = 0
    away_reds = 0
    home_off_subs = 0
    home_def_subs = 0
    away_off_subs = 0
    away_def_subs = 0

    for event in events:
        full_text = event.get_text(" ", strip=True)

        # 1. Time
```

Data Science Scraping Project - Code Appendix

```
time_match = re.search(r'^(\d+(?:\:\d+)?)', full_text)
event_time = time_match.group(1) if time_match else "0"

# 2. Score
score_match = re.search(r'(\d+)[\s-:~](\d+)', full_text)
if score_match:
    s1, s2 = score_match.group(1), score_match.group(2)
    if len(s1) <= 2 and len(s2) <= 2:
        current_home_score = int(s1)
        current_away_score = int(s2)

classes = event.get('class', [])
team = "Home" if 'a' in classes else "Away" if 'b' in classes else "Unknown"

is_red_card = 'red_card' in str(event) or ('yellow_card' in str(event) and
'yellow_red_card' in str(event))
if is_red_card:
    if 'yellow_card' not in str(event) or 'yellow_red_card' in str(event):
        if team == "Home": home_reds += 1
        elif team == "Away": away_reds += 1

base_event = {
    "Game_ID": custom_game_id,
    "Season": season,
    "Matchweek": matchweek,
    "Game_Type": game_type,
    "Time": event_time,
    "Team": team,
    "Home_Score": current_home_score,
    "Away_Score": current_away_score,
    "Home_Red_Count": home_reds,
    "Away_Red_Count": away_reds,
    "Home_Off_Sub_Count": home_off_subs,
    "Home_Def_Sub_Count": home_def_subs,
    "Away_Off_Sub_Count": away_off_subs,
    "Away_Def_Sub_Count": away_def_subs,
    "Match_URL": "placeholder"
}

# --- EVENT LOGIC ---

# A. GOAL
if 'goal' in str(event) and 'own_goal' not in str(event):
    player_links = event.find_all('a')
    if player_links:
        scorer = player_links[0].get_text(strip=True)
        row = base_event.copy()
        row.update({
            "Event_Type": "Goal",
```

Data Science Scraping Project - Code Appendix

```
        "Player_1": scorer, "Pos_1": player_map.get(scorer, "Unknown"),
        "Player_2": None, "Pos_2": None,
        "Note": "Goal"
    })
    events_data.append(row)

# B. OWN GOAL
elif 'own_goal' in str(event):
    player_links = event.find_all('a')
    if player_links:
        scorer = player_links[0].get_text(strip=True)
        row = base_event.copy()
        row.update({
            "Event_Type": "Own Goal",
            "Player_1": scorer, "Pos_1": player_map.get(scorer, "Unknown"),
            "Player_2": None, "Pos_2": None,
            "Note": "Own Goal"
        })
        events_data.append(row)

# C. RED CARD
elif is_red_card:
    if 'yellow_card' not in str(event) or 'yellow_red_card' in str(event):
        player_links = event.find_all('a')
        if player_links:
            player = player_links[0].get_text(strip=True)
            row = base_event.copy()
            row.update({
                "Event_Type": "Red Card",
                "Player_1": player, "Pos_1": player_map.get(player, "Unknown"),
                "Player_2": None, "Pos_2": None,
                "Note": "Red Card"
            })
            events_data.append(row)

# D. SUBSTITUTION
elif 'substitute' in str(event):
    player_links = event.find_all('a')
    if len(player_links) >= 2:
        p_on = player_links[0].get_text(strip=True)
        p_off = player_links[1].get_text(strip=True)
        pos_on = player_map.get(p_on, "Unknown")
        pos_off = player_map.get(p_off, "Unknown")

        is_off, is_def = determine_sub_type(pos_off, pos_on)

# UPDATE COUNTERS
if team == "Home":
    home_off_subs += is_off
```

Data Science Scraping Project - Code Appendix

```
        home_def_subs += is_def
    elif team == "Away":
        away_off_subs += is_off
        away_def_subs += is_def

    row = base_event.copy()
    row["Home_Off_Sub_Count"] = home_off_subs
    row["Home_Def_Sub_Count"] = home_def_subs
    row["Away_Off_Sub_Count"] = away_off_subs
    row["Away_Def_Sub_Count"] = away_def_subs

    # READABLE NOTE
    if is_off: note_text = "Offensive Sub"
    elif is_def: note_text = "Defensive Sub"
    else: note_text = "Neutral Sub"

    row.update({
        "Event_Type": "Substitution",
        "Player_1": p_off, "Pos_1": pos_off,
        "Player_2": p_on, "Pos_2": pos_on,
        "Note": note_text
    })
    events_data.append(row)

final_row = {
    "Game_ID": custom_game_id,
    "Season": season,
    "Matchweek": matchweek,
    "Game_Type": game_type,
    "Time": "FT",
    "Team": "N/A",
    "Event_Type": "Final Result",
    "Home_Score": current_home_score,
    "Away_Score": current_away_score,
    "Home_Red_Count": home_reds,
    "Away_Red_Count": away_reds,
    "Home_Off_Sub_Count": home_off_subs,
    "Home_Def_Sub_Count": home_def_subs,
    "Away_Off_Sub_Count": away_off_subs,
    "Away_Def_Sub_Count": away_def_subs,
    "Player_1": None, "Pos_1": None,
    "Player_2": None, "Pos_2": None,
    "Note": "Final Game State",
    "Match_URL": "placeholder"
}
events_data.append(final_row)

return events_data
```

Data Science Scraping Project - Code Appendix

```
def scrape_with_retry(url, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = requests.get(url, impersonate="safari15_3", timeout=15)
            if response.status_code == 200:
                return response
            elif response.status_code in [403, 429]:
                print(f"    !!! Blocked (403/429). Waiting 60s before Retry
#{attempt+1}...")
                time.sleep(60 + random.uniform(1, 10))
            else:
                print(f"    Error: {response.status_code}")
                return None
        except Exception as e:
            print(f"    Connection Error: {e}. Retrying...")
            time.sleep(10)
    print(f"    Failed after {max_retries} attempts. Skipping.")
    return None

# --- MAIN EXECUTION ---
try:
    df_links = pd.read_csv(input_file)
    all_data = []

    print(f"Starting scrape of {len(df_links)} matches...")

    current_season = ""
    current_mw = ""
    game_counter = 1

    for index, row in df_links.iterrows():
        url = row['Match_URL']
        print(f"Processing ({index+1}/{len(df_links)}): {url}")

        response = scrape_with_retry(url)

        if response and response.status_code == 200:
            try:
                soup = BeautifulSoup(response.content, 'html.parser')

                game_type = "Cup" if "cup" in url.lower() or "copa" in url.lower() else
"League"

                season, matchweek, uuid = get_match_metadata(soup, url)

                if matchweek != current_mw or season != current_season:
                    current_mw = matchweek
                    current_season = season
                    game_counter = 1
            else:
```


Data Science Scraping Project - Code Appendix

```
        game_counter += 1

    # --- NEW ID LOGIC: PREPEND "1" ---
    # Example: 120230101 (League 1, Year 2023, Week 01, Game 01)
    custom_id = f"4{season}{matchweek.zfill(2)}{str(game_counter).zfill(2)}"

    p_map = build_player_map(soup)
    match_events = parse_events(soup, p_map, game_type, custom_id, season,
matchweek)

    for event in match_events:
        event['Match_URL'] = url
        all_data.append(event)

    time.sleep(random.uniform(6, 10))

except Exception as e:
    print(f"    Error parsing data: {e}")

if (index + 1) % 20 == 0:
    print("    (Autosaving progress...)")
    temp_df = pd.DataFrame(all_data)
    temp_df.to_csv(output_file, index=False)

final_df = pd.DataFrame(all_data)
final_df.to_csv(output_file, index=False)
print(f"Done! Data saved to {output_file}")

except FileNotFoundError:
    print(f"Error: Could not find {input_file}. Did you run the Harvester for Premier
League?")
except KeyboardInterrupt:
    print("\nScript stopped by user. Saving what we have so far...")
    if 'all_data' in locals() and len(all_data) > 0:
        pd.DataFrame(all_data).to_csv(output_file, index=False)
        print(f"Saved {len(all_data)} rows to {output_file}")
```

